

CASE STUDY 3 - AI Engineering Evaluation, Export and Optimization

Introduction

This case study focuses on understanding what happens when a trained deep learning model is transferred across different inference backends and deployment formats, specifically PyTorch, ONNX, and TensorRT. The main learning outcome of this project was to analyze how model behavior changes when moving from its original training framework to optimized execution environments.

Throughout this process, I examined what may be lost during conversion, including potential differences in numerical precision, metric consistency, and inference behavior. At the same time, I investigated what is gained, particularly in terms of latency reduction and increased frames per second (FPS).

A central question guiding this study was the trade-off between accuracy and speed. By comparing performance across different frameworks, this project aimed to understand whether performance optimization comes at a cost and how these trade-offs affect real-world deployment decisions.

Model conversion is performed to run a trained model on different inference frameworks and deployment environments. When converting a model from PyTorch to formats such as ONNX or TensorRT, the goal is to improve performance and compatibility while preserving the model's original behavior. It is important to ensure that essential properties, such as detection accuracy and prediction consistency, are not affected during this process.

Parity checks are used to confirm that the converted model produces results consistent with the original implementation. This involves comparing predicted bounding boxes, confidence scores, and class labels. If these outputs remain consistent, the conversion can be considered reliable. Without such validation, performance improvements may come at the cost of reduced accuracy.

Latency metrics measure how long the model takes to process a single image. The mean latency provides the average processing time, while percentile values such as p50 and p95 indicate performance stability. The p95 value is especially important because it reflects potential latency spikes that may affect real-time applications.

For detection accuracy, mAP 0.50 and mAP 0.95 are commonly used evaluation metrics. mAP 0.50 considers a detection correct when the overlap between the predicted and ground-truth bounding boxes reaches at least 50%. In contrast, mAP 0.95 requires a much higher overlap threshold, making it more strict and sensitive to localization errors. Therefore, mAP 0.95 provides a deeper assessment of detection precision.

Experiments and Analysis

Throughout this study, I conducted two experiments using images from the COCO dataset. The first experiment focused on comparing inference latency across different environments. I selected ten images from the COCO dataset at 640×640 resolution and evaluated the model in both PyTorch and ONNX formats.

backend	images	imgsz	mean_ms	p50_ms	p95_ms	fps
pytorch	10	640	27.370534100009536	20.34275050004908	60.05212429995474	36.53564071296846
onnx	10	640	21.67051649998939	21.51936350003325	26.59139730003516	46.14564678236855

table of Pytorch and Onnx comparison

The results showed that ONNX achieved lower mean latency (21.67 ms) compared to PyTorch (27.37 ms). The p50 values were close, but a significant difference appeared in the p95 metric. PyTorch reached 60.05 ms at p95, while ONNX maintained a much lower value of 26.59 ms. We can confidently state that ONNX provided not only faster average inference but also more stable execution, with fewer latency spikes.

For the second experiment, my instructor suggested including TensorRT, as it is a widely used and highly optimized inference framework in deployment scenarios. The goal was to compare PyTorch, ONNX, and TensorRT under controlled conditions.

To isolate the model’s detection time, I used a single image and ran inference 1000 times on the same input. This approach helped eliminate external factors such as image loading and preprocessing overhead, allowing the benchmark to reflect only the model’s “forward pass” execution time.

The experiment was conducted on a Google Colab environment using an NVIDIA T4 GPU. To ensure fairness and reliability, each benchmark was repeated five times. As expected, the results varied slightly between runs due to GPU behavior, dynamic frequency scaling, and background processes. However, the overall performance trend remained consistent across all trials. This setup allowed for a clearer comparison of the pure inference performance of each framework.

Method	FPS	ms/img
ONNX	127.92171364688912	7.817281144000049
TensorRT	253.85150735378318	3.939310861000081
PyTorch	98.94850633200035	10.106266754999979

Method	FPS	ms/img
ONNX	126.67254775975556	7.89437030900001
TensorRT	257.06884814522846	3.890008483000088
PyTorch	100.47190796134565	9.953030855000067

Method	FPS	ms/img
ONNX	113.95801800269001	8.775161392999962
TensorRT	245.07062656444995	4.080456372999947
PyTorch	102.42509542664443	9.763232300000027

3 different executions of the 3 benchmarks(PyTorch, Onnx and TensorRT)

The benchmark was repeated multiple times to ensure consistency. Although the exact FPS values varied slightly between runs, the overall ranking remained stable across all experiments.

TensorRT consistently achieved the highest FPS and the lowest inference time per image. ONNX performed better than the original PyTorch implementation, while PyTorch showed the lowest overall throughput in this controlled setup.

Across all runs, TensorRT operated at approximately 245–257 FPS (≈ 3.9 – 4.1 ms per image), ONNX ranged between 113–128 FPS (≈ 7.8 – 8.8 ms), and PyTorch remained around 99–102 FPS (≈ 9.7 – 10.1 ms). Despite small fluctuations due to GPU runtime behavior, the performance gap between the three frameworks was clear and consistent.

These results confirm that TensorRT provides substantial inference acceleration compared to both PyTorch and ONNX under identical conditions.

Conclusion

This study explored the effects of converting a YOLOv8 model across different inference frameworks. The results showed that ONNX improved latency stability compared to PyTorch while maintaining prediction consistency. Further optimization with TensorRT provided the highest inference speed under controlled conditions.

Overall, the experiments demonstrated that model conversion can significantly improve performance without sacrificing accuracy. Evaluating both parity and latency metrics proved essential for understanding the trade-offs involved in deployment optimization.

<https://github.com/bgmgncr/AI-Engineering-Evaluation-Export-and-Optimization>